Optimized Translation of MATLAB to RISC-V Architecture for Brain-Computer Interfaces

Kenan Erol, Electrical Engineering and Computer Science, Yale University
Advisor: Rajit Manohar, Electrical & Computer Engineering, Yale University

Introduction

Fully implantable BCIs impose severe limitations on power dissipation (15 mW total) to prevent thermal damage to brain tissue. HALO addresses the ensuing real-time compute and power constraints through a design that combines application-specific hardware accelerators with a low-power RISC-V microcontroller equipped with RVV.

There exists a gap between clinical algorithm development in MATLAB and the HALO hardware requirements of vectorized C code. This research develops an end-to-end compilation pipeline to bridge this gap, combining a TypeScript-based compiler and an RVV-optimized C matrix library to translate MATLAB algorithms into power-efficient RISC-V binaries.

KEY RESULTS

> Newly vectorized functions run up to **95.43%** faster than their scalar counterparts

> **4** newly vectorized functions (transpose, ctranspose, mean, var)

> Improvement in the number of MATLAB functions passing tests went up (C accurate to the MATLAB by 5×10^{-10}): **68 → 84/88**

> Compiler improvements since 2022 have made the scalar implementations for certain functions faster by up to **218.5%**, narrowing the vectorization advantage

> **Up to 26.75%** improvement in massive workloads involving calls to vectorized functions 1-100 times

Vectorization

Vectorization through RVV replaces thousands of scalar loop iterations with a single vector instruction, increasing speed and allowing quicker transitions into low-power sleep states.

There are 30 pre-existing binary and unary vectorized functions in the codebase. Integer memory operations, logical non-int operations, and binary operations had shifts in their vec-ref ratios due to updates in the GCC compiler making the scalar reference faster by 29-218.5%. During the course of this research, four new functions (transpose/complex transpose, mean, variance) were vectorized, bringing the number of vectorized functions up to 34. Vec-ref improvements of these new functions are in the 70-95% range.

MATLAB-to-C Code Compiler

Raw MATLAB files are passed into ts-traversal. First, the TreeSitter and TreeSitter-Matlab parsing libraries create a concrete syntax tree (CST), capturing the complete syntactic structure of the input MATLAB file and its algorithms. Then, the engine performs type inference on dynamically typed MATLAB. The inference algorithms in ts-traversal traverse the CST to understand the maximal dimensions, memory allocation requirements, and fundamental data types of all MATLAB variables, mapping them to statically typed C constructs.

The output is standard C code, augmented with vectorized and scalar functions via the RISCV-Matrix library and explicitly built for the parallel execution capabilities of HALO's microcontroller.

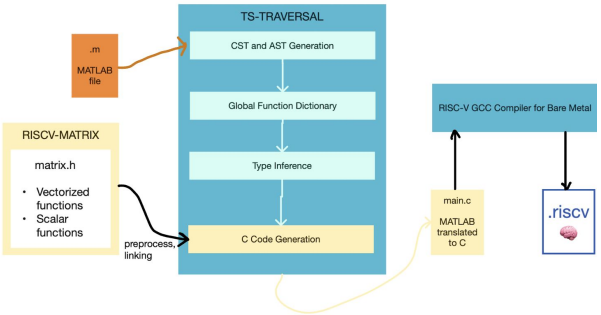

Figure 1: System architecture of the compilation pipeline.

Benchmarking

This research inherited ts-traversal (a TypeScript-based source-to-source compiler) and RISCV-Matrix (a C library that maps standard mathematical operations directly to RVV assembly instructions) from previous work done in the group.

Ts-traversal has 88 test kernels that compare C and MATLAB in heavy signal processing workloads (SPEED tests) and various linear algebraic/statistical tests. This research inherited the compiler with it passing 68/88 tests. Various debugging and matrix library fixes increased the passing test count to 84/88, with current failures in Octave and EEGLAB class support. RISCV-Matrix has two test suites - one for checking library correctness, and one for measuring vec-ref ratios. Full results are in Tables 1 and 2.

Test	int			double			complex		
	Grace	Cur.	Δ	Grace	Cur.	Δ	Grace	Cur.	Δ
abs	0.1932	0.1908	-0.0023	0.3897	0.3868	-0.0029	0.0044	0.0042	-0.0002
arg	0.0625	0.0732	+0.0107	0.0631	0.0740	+0.0109	1.0000	1.0000	+0.0000
ceil	0.6254	0.9103	+0.2849	0.0959	0.0942	-0.0017	0.0971	0.0939	-0.0032
ctranspose	—	0.2773	—	—	0.4356	—	—	0.4949	—
floor	0.6254	0.9103	+0.2850	0.0959	0.0942	-0.0017	0.0959	0.0922	-0.0037
full_duplicate	0.6254	0.9103	+0.2850	0.6253	0.9103	+0.2850	0.6253	0.6054	-0.0200
mean	—	0.0447	—	—	0.0830	—	—	0.1644	—
not	0.3550	0.3523	-0.0027	0.3024	0.8529	+0.5505	0.2994	0.8406	+0.5412
ones	0.3873	0.9225	+0.5352	—	—	—	—	—	—
round	0.6254	0.9103	+0.2850	0.1057	0.1148	+0.0091	0.0992	0.1070	+0.0078
transpose	—	0.2773	—	—	0.4356	—	—	0.6141	—
true	0.3411	0.3385	-0.0026	0.3130	0.8529	+0.5399	0.2899	0.8406	+0.5507
var	—	0.0765	—	—	0.1384	—	—	0.1092	—

Table 1: Vec/Ref ratios at N ~ 1,050,625 for unary operations.

Test	int × int			dbl × dbl			cpx × cpx		
	Grace	Cur.	Δ	Grace	Cur.	Δ	Grace	Cur.	Δ
and	0.6375	0.6314	-0.0061	0.4931	2.6775	+2.1844	0.3653	0.4173	+0.0519
divide	0.2802	0.8232	+0.5430	0.3249	0.3218	-0.0031	0.2351	0.1332	-0.1019
equal	0.1729	0.1384	-0.0345	0.1913	0.1502	-0.0411	0.1606	0.1289	-0.0317
isEqual	0.6711	0.6575	-0.0135	0.7971	0.7313	-0.0658	1.1719	1.1207	-0.0512
matlab_ge	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.2539	0.2510	-0.0029
matlab_gt	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.2539	0.2510	-0.0029
matlab_le	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.2774	0.2742	-0.0032
matlab_lt	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.2774	0.2742	-0.0032
minus	0.7619	0.7580	-0.0039	0.3249	0.3218	-0.0031	0.3921	0.4898	+0.0977
mtimes	0.0594	0.0481	-0.0113	0.0521	0.0401	-0.0120	0.0443	0.0318	-0.0126
ne	0.1632	0.1321	-0.0311	0.1799	0.1429	-0.0370	0.1529	0.1237	-0.0292
octave_ge	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.1416	0.1230	-0.0186
octave_gt	0.2817	0.2792	-0.0025	0.2533	0.8275	+0.5742	0.1416	0.1230	-0.0186
octave_le	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.1410	0.1222	-0.0188
octave_lt	0.2817	0.2792	-0.0025	0.2767	0.8275	+0.5508	0.1410	0.1222	-0.0188
or	0.6478	0.6416	-0.0062	0.5182	0.5107	-0.0075	0.4620	0.4475	-0.0145
pairwise_max	0.0011	0.0008	-0.0003	0.0040	0.0041	+0.0000	0.1347	0.1166	-0.0182
pairwise_min	0.0011	0.0008	-0.0003	0.0040	0.0041	+0.0000	0.1341	0.1158	-0.0183
plus	0.7619	0.7580	-0.0039	0.3249	0.3218	-0.0031	0.3921	0.4898	+0.0977
scale	0.8068	0.8039	-0.0029	0.3532	0.3505	-0.0027	0.3102	0.3072	-0.0030
times	0.7619	0.7580	-0.0039	0.3249	0.3218	-0.0031	0.3017	0.2973	-0.0044

Table 2: Vec/Ref ratios at N ~ 1,050,625 for binary operations

Workload Speed Ups and Downs

While most SPEED tests did not show improvement after further vectorization and C code generation correctness fixes, the transposeM vectorization did show notable improvement — 0.93% on 1000×1000 matrices and 26.75% on 100×100 matrices. While the numbers seem tame, they tell a bigger story of improvement due to the transposeM calls in these test cases being made 1 and 100 times respectively. This suggests that with further vectorization, even if a heavy math function is called only a few times, having it vectorized enables quicker transition into desired idle states for patient safety.

Some test kernels slowed down by up to 96% due to their print statement heavy nature of the tests. The definition of print was changed for increased precision in order to have accurate tolerance checking and get more tests passing.